MINIBANK

A Java-Based Account Management Simulation

PROJECT REPORT

Submitted by	Neev Patel
Student ID	25AIML054
Programme	B.Tech — Artificial Intelligence & Machine Learning
Language / Platform	Java • Console Application

Prepared for academic learning and demonstration

Project Overview

MiniBank is a menu-driven Java application that models a small set of everyday banking activities. It lets a user create account records, add money, withdraw money, move funds between two accounts, review the account directory, and inspect transaction messages recorded during the current run. The project is designed to practise programming fundamentals; it is not intended for real financial use.

1. Need for the Application

Account operations involve more than changing a number. A program should identify the correct account, check the amount entered, prevent a withdrawal that exceeds the available balance, and give the user clear feedback when an operation cannot be completed. MiniBank provides a small environment for exploring these checks through a straightforward console workflow.

2. Intended Outcomes

• Represent an account with an account number, holder name, and balance.

• Offer a menu for the main account operations and repeat it until the user exits.

• Validate names, account numbers, and currency input before processing a request.

• Prevent withdrawals and transfers when the source account has insufficient funds.

• Keep a time-stamped activity list during the active program session.

• Practise Java collections, methods, exception handling, date/time utilities, and numeric types.

3. Implementation Design

The application is organised around one main class, MiniBank, with a nested Account class. The Account object stores the account number, account holder, and balance. A LinkedHashMap associates account numbers with account objects, while an ArrayList stores readable transaction entries. A Scanner reads console input, and LocalDateTime with DateTimeFormatter supplies timestamps.

BigDecimal is used for money values so the program can work with decimal currency amounts without relying on binary floating-point arithmetic. The application currently keeps records in memory, so account data and the activity list are cleared when the program ends.

4. User Journey

Stage	What happens
Launch	The program prints the MiniBank heading and displays the main menu.
Choose an operation	The user selects account creation, deposit, withdrawal, transfer, account listing, history, or exit.
Check the request	The program looks up the account and validates the amount or other required input.
Apply the operation	A balance is changed only after the relevant checks pass.
Record and respond	The program prints a result message and records the activity where applicable.

5. Input Rules and Transaction Checks

• The account holder name must not be blank and must be no longer than 60 characters.

• Currency input must be numeric and contain no more than two decimal places.

• A deposit, withdrawal, or transfer must be greater than zero; an opening deposit may be zero.

• An unknown account number is reported instead of being used.

• A withdrawal is cancelled if the account does not have enough money.

• A transfer requires two different existing accounts and sufficient funds in the source account.

• Non-integer menu input is handled with a message so the loop can continue.

6. Verification Plan

The following cases should be run in a Java environment before final submission. Mark the outcome only after trying the case and observing the program's response.

Test case	Expected behaviour	Observed result
Create an account with a valid name and opening amount	Account number should be assigned and shown	To be completed
Enter a blank account holder name	Request should be rejected	To be completed
Deposit a positive amount into an existing account	Balance should increase by the entered amount	To be completed
Withdraw less than or equal to the available balance	Valid withdrawal should reduce the balance	To be completed
Attempt to withdraw more than the available balance	Withdrawal should be cancelled	To be completed
Transfer money between two different accounts with enough funds	Source decreases and destination increases	To be completed
Transfer money to the same account	Transfer should be rejected	To be completed
Enter letters for a menu choice or malformed amount	Input error should be reported without an uncaught crash	To be completed
Exit and restart the program	Current in-memory account records will not be retained	To be completed

7. Known Boundaries

• Data is stored only in memory and disappears when the application closes.

• The program is a local console demonstration and does not connect to a bank or payment service.

• Authentication, role permissions, durable storage, concurrency controls, and production security are outside the current scope.

• The activity history is a list of descriptions rather than a formal ledger or database transaction log.

8. Possible Extensions

• Persist accounts and transactions in a file or database.

• Add account search and a clearer formatted statement for a selected account.

• Create JUnit tests for deposits, withdrawals, transfer rules, and invalid input.

• Develop a Java Swing interface while keeping business rules separate from display code.

• Improve transfer consistency and design more structured transaction records.

Conclusion

MiniBank demonstrates how a small Java program can organise account-related operations using objects, collections, validation, exception handling, and decimal arithmetic. Its most important learning point is that a transaction should be checked before changing an account balance. The current version is a learning prototype; further testing and persistent storage would be needed before extending its capabilities.

Student Review

Before submitting this report, review the implementation, run the test cases available to you, and update the observed-result column honestly. Add any college-specific course, semester, faculty, or submission-date details required by your department.